

Senior DevOps / SRE Interview Questions & Answers — Part 2

This document continues the remaining questions collected from the screenshots.

The answers are intentionally written in the tone expected from an experienced 8+ years DevOps/SRE engineer working in production-scale environments.

36. Pod is running but returning 503. How do you debug?

Answer

A 503 usually means upstream service unavailable.

My approach:

1. Verify pod readiness
2. Check readiness probe failures
3. Validate service endpoints
4. Confirm ingress/backend mapping
5. Check application dependency health
6. Review ingress controller logs
7. Validate network policies

Commands:

```
kubectl get endpoints
kubectl describe svc
kubectl describe ingress
```

Most common root causes:

- readiness probes failing
 - selector mismatch
 - upstream dependency failure
 - ingress routing issues
-

37. How does Kubernetes scheduling work internally?

Answer

The scheduler watches unscheduled pods and selects the best node.

Scheduling phases:

1. Filtering
2. resource availability
3. taints/tolerations
4. node affinity
5. topology constraints
6. Scoring
7. least utilized node
8. spread across zones
9. affinity preferences
10. Binding
11. pod assigned to node

Common scheduling failures:

- insufficient CPU/memory
- PVC pending
- taints
- node selectors mismatch
- resource fragmentation

In production, I monitor scheduler latency and cluster fragmentation closely.

38. Explain CNI plugins and cross-node pod communication.

Answer

CNI plugins provide pod networking.

Typical flow:

1. Pod gets virtual interface
2. IP assigned from CNI
3. Routes configured
4. Traffic forwarded across nodes

Common CNIs:

- Calico
- Cilium
- Flannel
- Weave

Cross-node communication happens using:

- VXLAN
- BGP
- eBPF
- overlay networks

At scale, I prefer Cilium because:

- eBPF performance
- observability
- advanced security policies

39. How do you debug intermittent pod restarts?

Answer

I start with:

```
kubectl describe pod
kubectl logs --previous
```

Then validate:

- OOMKilled
- probe failures
- node instability
- application crashes
- kernel pressure

I correlate with:

- deployments
- autoscaling events
- traffic spikes
- node rotations

Intermittent restarts usually require historical metrics and event correlation.

40. CI pipeline builds 50 images and takes 20 minutes. How would you optimize?

Answer

I optimize both build architecture and pipeline parallelism.

Strategies:

- parallel image builds
- Docker layer caching
- multi-stage builds
- incremental builds
- shared base images
- dependency caching
- distributed runners

I also remove unnecessary scans from critical path.

In large environments, proper caching alone reduces build time dramatically.

41. How do you design a secure CI/CD pipeline?

Answer

Security must exist in every stage.

My pipeline includes:

Source stage

- branch protection
- signed commits
- secret scanning

Build stage

- SAST
- dependency scanning
- SBOM generation

Container stage

- image scanning
- non-root containers
- minimal base images

Deployment stage

- RBAC
- approval gates
- policy enforcement
- signed artifacts

Runtime

- Falco
- runtime security
- audit logging

Security should shift left without blocking developer productivity.

42. Pipeline works in staging but fails in production. How do you debug?

Answer

This is usually environment drift.

I compare:

- secrets
- IAM permissions
- DNS
- network rules
- quotas
- resource limits
- certificates

I also verify:

- production-only dependencies
- feature flags
- traffic volume differences

The goal is identifying environmental differences systematically.

43. Design a multi-region highly available architecture.

Answer

My design includes:

- active-active or active-passive setup
- multi-AZ clusters
- global load balancing
- replicated databases
- automated failover
- infrastructure as code
- centralized observability

Critical considerations:

- RPO/RTO
- consistency model
- latency
- failover automation

I always test disaster recovery regularly.

44. AWS bill increased 3x overnight. How do you investigate?

Answer

First I identify which service caused the spike.

Using:

- Cost Explorer
- CUR reports
- CloudWatch metrics

Common causes:

- traffic surge
- NAT gateway usage
- runaway autoscaling
- excessive logs
- data transfer
- forgotten resources

Then I:

- identify ownership
 - contain cost quickly
 - apply tagging improvements
 - add budget alerts
-

45. How do you manage infrastructure drift in Terraform?

Answer

I prevent manual changes first.

My strategy:

- GitOps workflow
- restricted console access
- drift detection pipelines
- scheduled terraform plan checks
- policy enforcement

If drift occurs:

1. Identify differences
2. Determine authorized vs unauthorized changes
3. Reconcile safely
4. Update code or revert infra

Drift management is mostly governance.

46. How do you design secrets management across 100+ microservices?

Answer

I centralize secrets using:

- Vault
- AWS Secrets Manager
- External Secrets Operator

Principles:

- short-lived credentials
- dynamic secrets
- least privilege
- automatic rotation
- audit logging

I avoid storing secrets directly in:

- Git
- images
- environment files

At scale, secret rotation automation becomes critical.

47. System passed scans but still got compromised. What security layers were missing?

Answer

Scans alone are not security.

Possible missing controls:

- runtime protection
- behavioral monitoring
- zero trust networking
- IAM hardening
- least privilege
- supply chain security
- anomaly detection
- patch management

I always assume preventive controls will eventually fail.

Detection and response are equally important.

48. How do you design observability for distributed systems?

Answer

Observability must include:

- metrics
- logs
- traces
- events

My stack typically includes:

- Prometheus
- Grafana
- Loki/ELK
- Tempo/Jaeger

Key principles:

- correlation IDs
- SLO-driven dashboards
- actionable alerts
- low cardinality metrics
- standardized logging

Good observability reduces MTTR significantly.

49. How do you implement SLO-based alerting without alert fatigue?

Answer

I alert on user impact, not infrastructure noise.

Strategies:

- SLO-based alerts
- burn-rate alerts
- alert grouping
- deduplication
- severity classification
- symptom-based alerting

Bad alerts create operational blindness.

I prioritize signal quality over alert quantity.

50. Service latency spikes every 60 seconds. How do you debug?

Answer

Periodic spikes usually indicate scheduled behavior.

I investigate:

- cron jobs
- GC cycles
- autoscaling sync loops
- metrics scraping
- backup jobs
- health checks

I correlate latency graphs with infrastructure events.

Then use tracing to isolate the exact component causing periodic degradation.

51. How do you handle Terraform state locking in a large team?

Answer

I use remote backends with locking.

Example:

- S3 backend
- DynamoDB lock table

Best practices:

- separate state files
- modular architecture
- environment isolation
- CI-controlled applies
- least privilege access

State corruption prevention is more important than recovery.

52. How do you detect and automatically remediate configuration drift?

Answer

I combine:

- GitOps
- policy enforcement
- scheduled reconciliation
- drift detection pipelines

Tools:

- ArgoCD
- Flux
- Terraform Cloud
- OPA/Gatekeeper

Auto-remediation works well only for well-defined changes.

Critical production environments still require approval gates.

53. How do you import 100+ manual resources into IaC safely?

Answer

I never bulk import blindly.

Approach:

1. Inventory existing resources
2. Group dependencies
3. Import incrementally
4. Validate plans after every import
5. Lock production changes during migration

Key objective:

Avoid unintended recreation.

54. Describe packet flow from browser to pod behind ingress.

Answer

Flow:

1. DNS resolution
2. Request hits load balancer
3. Ingress controller receives traffic
4. Ingress routes request to service
5. Service forwards to pod endpoint
6. Pod processes request

Debugging layers:

- DNS
- LB
- ingress
- service
- pod
- application

This layered model is essential during outages.

55. How do you implement least privilege at pod level?

Answer

I minimize permissions aggressively.

Controls:

- dedicated service accounts
- RBAC restrictions
- runAsNonRoot
- readOnlyRootFilesystem
- seccomp profiles
- dropped Linux capabilities
- network policies

Security must be enforced by default through policy.

56. How do you troubleshoot exit code 137?

Answer

Exit code 137 usually means SIGKILL.

Most common cause:

- OOMKilled

I verify:

```
kubectl describe pod
```

Then analyze:

- memory limits
- heap usage
- memory leaks
- node pressure

Mitigation:

- optimize application
 - increase memory
 - adjust JVM settings
 - scale horizontally
-

57. What are the Four Golden Signals?

Answer

The Four Golden Signals are:

1. Latency
2. Traffic
3. Errors
4. Saturation

These provide a strong operational health model.

I design dashboards and alerts around these signals first.

58. What is Error Budget?

Answer

Error budget represents acceptable unreliability.

Formula:

If SLO = 99.9% Then allowable failure = 0.1%

Error budgets help balance:

- reliability
- feature velocity

If the budget is exhausted, deployment velocity should slow down.

59. How do you reduce high cardinality in Prometheus?

Answer

High cardinality destroys Prometheus performance.

I avoid labels like:

- request_id
- session_id
- user_id

Strategies:

- label standardization
- aggregation
- metric relabeling
- retention tuning
- histogram optimization

Observability systems must scale sustainably.

60. How do you prevent CI/CD secret leakage during malicious PRs?

Answer

I isolate untrusted code execution.

Controls:

- no secrets for fork PRs
- approval workflows
- isolated runners
- masked secrets
- short-lived credentials
- restricted permissions

Supply chain security is critical in modern CI/CD.

61. Blue-Green vs Canary — when does one fail while the other succeeds?

Answer

Blue-Green:

- instant switch
- easier rollback
- higher infra cost

Canary:

- gradual rollout
- lower blast radius
- better production validation

Blue-Green fails when:

- hidden production-only issues exist

Canary succeeds because:

- partial traffic exposure detects problems early

Choice depends on risk tolerance.

62. How do you ensure artifact integrity between QA and production?

Answer

The exact same artifact must move through environments.

Controls:

- immutable artifacts
- signed images
- digest pinning
- SBOM validation
- provenance tracking

I never rebuild separately for production.

63. How do you handle blame-game situations between Dev and Ops?

Answer

I focus discussions on systems, not individuals.

My approach:

- establish shared facts
- create incident timeline
- define action items
- improve observability
- improve ownership clarity

Healthy incident culture is critical for high-performing teams.

64. What is your 30-60-90 day plan for migrating CI/CD platforms?

Answer

First 30 days

- understand pipelines
- inventory integrations
- identify risks

Next 60 days

- migrate low-risk workloads
- create reusable templates
- validate security

Final 90 days

- migrate critical systems
- optimize performance
- decommission legacy platform

Migration success depends more on planning than tooling.

65. How do you secure database credential rotation in Kubernetes?

Answer

I automate rotation using:

- Vault dynamic secrets
- External Secrets Operator
- AWS Secrets Manager rotation

Applications should reload credentials dynamically whenever possible.

Downtime-free secret rotation is a critical production capability.

66. What is your backup and restore strategy for Kubernetes?

Answer

I back up:

- etcd
- manifests
- persistent volumes
- secrets

Tools:

- Velero
- cloud snapshots
- GitOps repositories

Most important:

Regular restore testing.

Backups without restore validation are unreliable.

67. How do you implement autoscaling for fluctuating traffic?

Answer

I combine:

- HPA
- Cluster Autoscaler
- predictive scaling
- queue-based scaling

Metrics:

- CPU
- memory
- RPS
- queue depth

- latency

Autoscaling must be tested under load before production.

68. A node suddenly dies. What happens to the pods?

Answer

Kubernetes marks node NotReady.

Then:

1. Pods become unreachable
2. Controller reschedules replicas
3. Traffic reroutes automatically
4. Stateful workloads recover based on storage strategy

Recovery time depends on:

- readiness probes
 - PDBs
 - storage class
 - failover configuration
-

69. Service not accessible externally. Where do you start?

Answer

I debug layer by layer:

1. Pod reachable?
2. Service endpoints healthy?
3. Ingress configured?
4. Load balancer healthy?
5. DNS resolving?
6. Firewall/security groups open?

Structured isolation is the fastest troubleshooting approach.

70. How do you optimize resource requests and limits?

Answer

I base requests on real usage metrics.

Too low:

- throttling
- instability

Too high:

- wasted capacity
- scheduling failures

I use:

- VPA recommendations
- historical metrics
- load testing
- performance profiling

Efficient resource tuning saves significant cloud cost.

Final Note

Senior DevOps/SRE interviews are not testing only commands.

They evaluate:

- production judgment
- RCA thinking
- communication during incidents
- system design maturity
- reliability mindset
- operational calmness under pressure

The strongest candidates think in terms of:

Impact → Stabilization → Isolation → Root Cause → Prevention